HUSH: Provably Safe Sub-Block Hibernation for Real-Time Event-Driven SNN Inference on FPGAs

Anubhav Jha and Rajat Subhra Chakraborty [spelling to confirm]
Department of Electrical Engineering / Computer Science and Engineering, IIT Kharagpur, India [affiliations to confirm]

***Abstract*—In a fully connected spiking layer, most of the time and energy goes into fetching the weights of every neuron for every input spike, although few neurons fire. We present HUSH, an output-side skipping scheme that puts a sub-block of eight neurons to sleep for a timestep only when a cheap integer envelope proves that none of them can reach the threshold, so the gated update never suppresses a spike that the ungated update from the same state would produce. The envelope combines the exact maximum membrane potential of the sub-block with a per-input proxy table; it is computed while the previous timestep is processed, and the proof is checked formally on the RTL arithmetic. Trained with the gate in the loop, a 578-256-10 integer network on N-MNIST runs on a Xilinx Artix-7 XC7A100T at 100 MHz entirely on chip, from raw sensor events to the class: at 95.74 % accuracy it is 2.44× faster than the same hardware with the gate off (115 µs vs. 280 µs per inference; 98.21 % accuracy) with 57 % less energy (18.4 vs. 42.5 µJ), and at 96.54 % 1.75× faster with 44 % less energy. On the same chip and weights as BASE, the open Spiker+ generator needs 818 µs and 255 µJ (6.0× BASE). In RTL simulation of a live event stream, every 3 ms window is processed within 4.5 µs of its end, and the design sustains up to 1.33–2× the event rate of the ungated design with the same 1,024-event input buffer.**

I. Introduction

Event-driven spiking neural networks (SNNs) are attractive for always-on sensing because their input is sparse and their arithmetic is additive. On FPGAs, accelerators usually exploit *input* sparsity: work is done only for input spikes that occur. For a fully connected layer with N neurons, each input spike still costs N weight reads from block RAM, although in a given timestep only a few percent of the neurons fire. Skipping the *output* side, i.e. neurons that will not fire, is the remaining large saving, but it is hard: whether a neuron fires depends on the very weights one wants to skip. Published output-side schemes are heuristics (they predict that a neuron stays silent and can be wrong) and therefore change the network's behaviour in ways that are hard to bound.

We make output-side skipping *provably safe*. Our contributions:

1) **HUSH gate:** an integer envelope per sub-block of eight neurons that upper-bounds every member's next membrane potential from the sub-block's exact current maximum and a per-input proxy table; a sub-block sleeps only if the bound is below threshold (Theorem 1), so at every timestep the gate removes no spike that the ungated update from the same state would produce. The safety arithmetic is proved formally (unbounded PDR) on the RTL modules.
2) **Architecture:** an envelope unit overlapped with the processing of the previous timestep, per-sub-block weight-column RAM enables, a whole network on chip (hidden layer, output layer overlapped one timestep behind, and an integer class decision), and a stream front-end that accepts raw DVS events.
3) **Evaluation on one chip:** bit- and cycle-exact RTL against a golden model; gate-level activity-based power; the same RTL with the gate off as baseline (BASE); Spiker+ [1] on the same device with BASE's weights; three training seeds per point; and real-time operation on a live N-MNIST stream.

II. Background and Related Work

Integer LIF. Each neuron keeps a 16-bit saturating potential V. Per timestep t, $U = \mathrm{sat}(L(V) + \sum_{j \in A_t} w_j)$ with leak $L(V) = V - (V \gg k)$, $k = 4$, INT8 weights and A_t the active inputs; the sum is exact before the single saturation. It spikes, $s = [U \geq \theta]$ ($\theta = 64$), and keeps $V' = U - \theta s$ (soft reset).

FPGA SNN accelerators on N-MNIST. Table III lists the designs we found that report N-MNIST results on an FPGA. They differ in network (convolutional, recurrent, fully connected), device and power method; two report no latency.

Output-side skipping. Three recent designs skip work for neurons predicted not to fire: Early [2] stops accumulation for neurons whose partial sum of the largest weights stays below a tuned threshold; SITRA [3] scores each output of a spiking transformer with a weight-MSB dot product and skips the weight fetches of outputs it predicts silent, overlapping detection with the previous token; PS4 [4] speculates over a time window and recomputes failed speculations. All three are heuristics with tuned thresholds and report 0.5–2 % accuracy loss from mispredictions, and all are evaluated in ASIC simulation. SPARSH [5] skips zero weights, zero spikes and refractory neurons; C-STEP [6] and the early-stop of [7] end inference early in time. HUSH differs in that its prediction is a proven bound: a sleeping sub-block cannot contain a neuron that would fire at that timestep, the bound carries membrane state across timesteps, and its granularity matches a block-RAM column, so a skip removes whole memory reads on an FPGA.

III. The HUSH Gate

Neurons are grouped into sub-blocks c of $b = 8$. Two tables are derived offline from the weights: the proxy $P_{jc} =$

$\max_{i\in c} W_{ji}$ and the mean $Q_{jc} = \lfloor \mathrm{mean}_{i\in c} W_{ji} \rfloor$. The hardware keeps $R_c = \max_{i\in c} V_i$ exactly after every timestep. Before the weights of timestep t are fetched, the envelope unit accumulates $D_c = \sum_{j\in A_t} P_{jc}$ and evaluates

$$E_c = \mathrm{sat}\big(L(R_c) + \max(0, D_c)\big), \qquad g_c = [E_c \geq \theta]. \quad (1)$$

Theorem 1 (Safety)**.** *If $g_c = 0$, every neuron i in sub-block c has $U_i < \theta$ in the ungated update from the same state.*

Proof. L is monotone, so $L(V_i) \leq L(R_c)$; $W_{ji} \leq P_{jc}$ gives $\sum_{j\in A_t} W_{ji} \leq D_c \leq \max(0, D_c)$; and sat is monotone. Hence $U_i \leq E_c < \theta$. □

A sleeping sub-block skips its weight reads. Its neurons are updated with a shared mean-field drive, $U_i = \mathrm{sat}(L(V_i) + \sum_{j\in A_t} Q_{jc}) \leq E_c < \theta$, so they cannot spike either, and R_c stays exact. The gated layer therefore emits exactly the spikes of the ungated layer *from the same state*; the asleep state evolves approximately, so end-to-end accuracy is the measure of the approximation. We train with the gate in the loop (straight-through estimator) and add two penalties to the cross-entropy: the envelope's mean excess over the threshold, $\lambda_e \,\mathrm{mean}_{c,t} \max(0, E_c - \theta)/\theta$, and the expected positive proxy drive per timestep, $\lambda_d \,\mathrm{mean}_c \sum_j r_j \max(0, P_{jc})/\theta$ with r_j the firing rate of input j, which shrinks the largest weight of each sub-block. The three points use $(\lambda_e, \lambda_d) = (0.25, 0.1)$ gentler, (0.5, 0.25) gentle and (2, 1) aggressive; larger weights put more sub-blocks to sleep at some cost in accuracy.

IV. Architecture

Hidden layer. 256 neurons form 8 blocks of 32 lanes (4 sub-blocks of 8). The weight memory is four 64-bit column RAMs, one per sub-block; a column is read only if its sub-block is awake. Per timestep the sequencer combines the envelope bank (gates), then visits each block: an awake block streams the timestep's events through a 4-stage pipeline (event read, index register, weight read, accumulate) and finalises its lanes in two stages; an asleep block takes 3 cycles (state read, mean-field finalise, write-back). The envelope unit for timestep $t+1$ runs during the block loop of t (double-buffered banks). The cycle cost of a timestep with $n > 0$ events is $1 + \sum_b c_b + 2$ with $c_b = 4 + (n + 4)$ for an awake block and 3 for an asleep one, plus any wait for the envelope bank. Time is therefore saved per 32-neuron block (all four sub-blocks asleep), while energy is also saved per 8-neuron sub-block, whose weight column is not read.

Whole network. Hidden-layer spikes enter a 32-entry FIFO; a drain converts each set bit into an event of the output layer (32 lanes, 10 used, dense), which runs one timestep behind the hidden layer and overlapped with it. The class is the integer argmax of spike counts, ties broken by the summed pre-reset potential, then by index.

Stream front-end. Raw DVS events (x, y, p) are pooled 2×2 into the 578 inputs, de-duplicated per window (the model's input is a 0/1 raster), and written into a ring of 1,024 events (one RAMB18); an end-of-window marker closes the window, writes its range and lets the hidden layer start its envelope pass for that window. Markers have reserved FIFO space, so overload can only drop events. In stream mode the whole network takes 8,736 LUTs / 48.5 BRAMs (BASE) and 14,606 / 62.5 (HUSH), both at 100 MHz.

TABLE I
Accuracy and latency, whole network on chip (mean ± std, 3 seeds; latency over 500 test samples).

Design	Acc. (%)	Reads[c]	Lat. (µs)	Speedup[e]
BASE (gate off)[b]	98.21±0.13	1.000	279.5	1.00×
HUSH gentler	96.54±0.07	0.204	159.8±8.1	1.75×
HUSH gentle	95.74±0.07	0.118	114.9±7.0	2.44×
HUSH aggressive	92.32±0.68	0.070	82.9±7.8	3.39×

[b] trained as long as the gated nets (10 + 12 epochs); 10 epochs only: 97.66±0.07. [c] weight-column reads relative to BASE. [e] mean of the per-seed ratios.

V. Verification

A golden integer model (Python) defines the behaviour. The RTL matches it bit for bit on 80 randomised layer configurations (gate, mean-field, sizes, leak, threshold, load port) and, for the whole network, on 50 N-MNIST test samples for each of four weight sets: every hidden and output block result and every class, with cycle counts equal to an exact cycle model. Forty random whole networks (including a 4-entry spike FIFO that forces back-pressure) are also bit-exact. The stream mode is bit-exact in every run without dropped events, at true sensor timing and up to 2,000× replay. The safety arithmetic (envelope, lanes, exact R recursion) of one sub-block, at reduced widths, is proved for all reachable states by ABC PDR in SymbiYosys for both sleep semantics (memories and sequencing are covered by the simulations above); a mutant with an unsafe proxy fails immediately. Gate-level simulation of the routed netlists reproduces the model's cycle count for every inference (+3 cycles of restart).

VI. Results

Setup. XC7A100T-1CSG324C, Vivado 2024.1, 100 MHz. Power from gate-level switching activity (SAIF) of 20 test inferences per design on the routed netlist; energy per inference = power × latency. Accuracy on the full 10,000-sample N-MNIST test set ($T = 100$ windows of 3 ms) with the hardware's integer decision rule; accuracy and latency over three training seeds; area, power and energy for the seed-0 networks on 20 samples. BASE's power and energy were measured with its own (22-epoch) weights; they equal those of the 10-epoch BASE (152 mW, 42.5 µJ).

Accuracy–savings front. Table I. The gated networks are fine-tuned for 12 epochs from the 10-epoch ungated network; the same fine-tuning without the gate raises the ungated network from 97.66 % to 98.21 %, so we use that network as BASE. Against it the gate costs 1.7–5.9 points and reads 7–20 % of the weight columns. Training longer with the gate (12 instead of 6 epochs) improved every axis of the gentle point (95.37 %, 1.98× → 95.74 %, 2.44×). In a single-seed

TABLE II
HARDWARE ON XC7A100T, 100 MHz (WHOLE NETWORK, 20 ON-CHIP SAMPLES).

Design	LUT	BRAM	WNS (ns)	Lat. (µs)	Power (mW)	Energy (µJ)
BASE	7,745	80	+0.956	279.5	152	42.5
HUSH gentler	13,885	95	+0.450	151.2	157	23.7
HUSH gentle	13,885	95	+0.450	115.3	160	18.4
HUSH aggr.	13,885	95	+0.450	87.7	164	14.4
Spiker+ [1]	13,548[a]	70.5	+0.005[a]	818.4	311	254.5

Latency and power over the 20 simulated inferences; static power 101–102 mW.
[a] high-effort implementation; power measured on the default-effort netlist (WNS −0.241 ns).

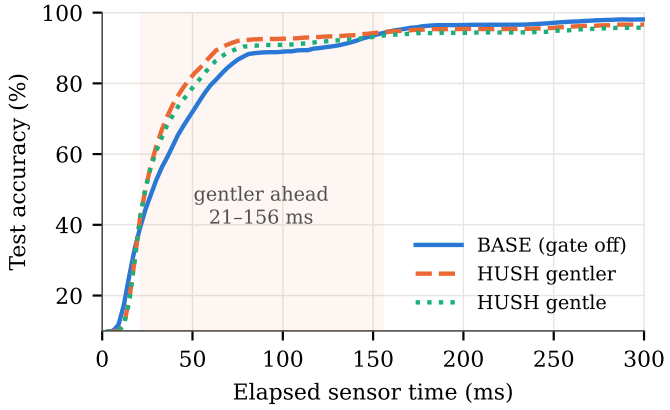


Fig. 1. Accuracy when the class is read after the first windows of a recording (10,000 test samples). BASE is trained as long as the gated networks.

test, a wider hidden layer (384) raised the ungated accuracy to 98.0 % but not the gated one (95.9 %, 6 epochs).

Same chip. Table II. HUSH adds 6.1k LUTs and 15 BRAMs (envelope unit, proxy and mean tables, sub-block control). Spiker+ (clock-driven, ungated) runs BASE's 10-epoch INT8 weights (97.53 % on its neuron semantics, fixed point with 6 fractional bits), scans all 578 inputs serially each timestep and needs 81.8k cycles per inference in our harness (81.4k for the bare generated network). With default implementation settings it misses 100 MHz by 0.24 ns; with high-effort directives it meets 100 MHz (+0.005 ns, 13,548 LUTs). Its power was measured on the default-effort netlist; the high-effort one is estimated 4 % higher, so the reported Spiker+ energy is a lower bound. With the gate, HUSH spends 44–66 % less energy per inference than BASE although its power is 3–8 % higher: the extra envelope logic costs less than the time it saves. Spiker+ draws 311 mW, 140 mW of it in block RAM, which it reads every cycle (ours: 12 mW, read only for awake sub-blocks), and needs 6.0× the energy of BASE and 13.8× that of HUSH gentle. Energies in Table II use the latency of the same 20 inferences (e.g. gentler 151.2 µs vs. 159.8 µs over 500 samples); the speedups of Table I use the 500-sample means.

Heuristic gates. On the 6-epoch gentle network and 200 test samples, we replaced only the predictor and bisected each heuristic's threshold to HUSH's read fraction (0.157): a state-free proxy inspired by SITRA (count of negative weights of the active inputs; not SITRA's MSB dot product) reaches 66.0 % and suppresses 15,121 spikes; an Early-style predictor ($L(V)$ plus the top-8 active weights) 93.5 % with 36; HUSH 95.5 % with none. Two caveats favour HUSH: the network was trained with the HUSH gate, not with the heuristics, and the Early-style predictor's own weight reads are not charged.

VII. REAL-TIME OPERATION

N-MNIST recordings deliver on average 41.9 raw events per 3 ms window (26.5 distinct pooled inputs; maximum 96). **At true sensor timing** (RTL simulation of 4 recordings per design, bit-exact, no drops) a window is processed in 1.47 µs on average with HUSH (3.04 µs BASE), at most 4.50 µs (6.64 µs), i.e. $\leq$ 0.22 % of the window; the class is ready 0.89 µs (2.03 µs) after the last window; the input ring never holds more than 127 events. HUSH shortens the average window (by 37 %), not the busiest one, where most blocks wake up (840 vs. 828 cycles). **Energy in real-time operation.** At sensor speed the chip idles more than 99.9 % of the time, so a 300 ms recording costs ≈30.3 mJ of static energy (101 mW) for either design and HUSH saves only its dynamic share (6.7 vs. 14.3 µJ per inference). HUSH's gain appears when the chip is kept busy, e.g. serving recordings faster than real time: at the highest replay speed that drops nothing on the heaviest recordings (750× BASE, 1,000× HUSH gentle) a recording costs 152 mW×0.40 ms = 61 µJ with BASE and 160 mW×0.30 ms = 48 µJ with HUSH. **Faster than real time.** Replaying the 20 test recordings with the most events (1.7× the mean rate) with the same 1,024-event buffer, BASE drops events above 750× real time, HUSH gentle above 1,000× on all 20 recordings (23.7 M events/s, 1.33×) and above 1,500× for the median recording (2.0×). **Early decisions.** Because the class can be read after any window, Fig. 1 reports accuracy vs. elapsed sensor time. Between 21 ms and 156 ms the gentler network is more accurate than BASE trained equally long (e.g. 87.7 % vs. 79.5 % at 60 ms and 92.6 % vs. 88.9 % at 99 ms); BASE leads from 159 ms on. The gated networks do not spike earlier or more (fewer output spikes in the first 99 ms: 39–41 vs. 52) but their early leading class is wrong less often (8.5–8.8 % vs. 10.6 % of 1,000 samples at 99 ms). The gentle network leads from 21 ms to 144 ms. An application that must decide within ≈140 ms is therefore both faster and more accurate with either gated network. (Spike statistics: first 1,000 test samples.)

VIII. COMPARISON WITH THE STATE OF THE ART

HUSH runs 37–435× faster per inference than the designs that report latency and needs 77–950× less energy per inference (18.4–23.7 µJ vs. ≈1.83 and ≈17.5 mJ computed as power × latency), on a smaller device with 4–6× fewer LUTs than the two designs that report them (3–12× fewer than Wu et al.'s range). Accuracy is comparable to those two designs (96.54 vs. 96.4 and 95.76 %) and 1.6–1.9 points below the two

TABLE III
FPGA SNN ACCELERATORS ON N-MNIST.

Design	FPGA	Acc. (%)	LUT	Power	Latency
Cheng et al. [7]	ZCU104	98.11	81.3k	0.81^{d}	n/r
FeNN-DMA [8]	KV260	98.46	55.0k	0.53^{d}	n/r
Surrogates [9]	xcvu9p	96.4	n/r	0.31	5.9 ms
Wu et al. [10]	ZCU102	95.76	39.7–165k	0.33–0.37	50 ms
HUSH gentler	XC7A100T	96.54	13.9k	$0.16/0.056^{d}$	0.16 ms
HUSH gentle	XC7A100T	95.74	13.9k	$0.16/0.058^{d}$	0.11 ms

Power in W; ^{d} dynamic only; n/r: not reported.

larger designs; with the gate off, the same hardware reaches 98.21 % at 280 µs. Comparability is limited: devices, networks and power methods differ, and our power is a Vivado estimate from gate-level activity, not a board measurement.

IX. LIMITATIONS AND CONCLUSION

The speedup depends on stream sparsity: per window it falls from 2.2× with fewer than 10 active inputs to 1.6× with more than 60, as more sub-blocks wake up, and it tends to 1× on dense streams. Serving many streams on one chip is limited by state memory, where BASE would fit more streams than HUSH. The gate costs accuracy, and power is estimated, not measured. HUSH shows that output-side skipping in SNN hardware can be both effective and provably safe; extending the envelope to convolutional layers and denser event streams (DVS-Gesture) is future work.

REFERENCES

[1] A. Carpegna, A. Savino, and S. Di Carlo, "Spiker+: a framework for the generation of efficient spiking neural networks FPGA accelerators for inference at the edge," arXiv:2401.01141, 2024.

[2] "Early: an importance-aware early firing and exit for SNN acceleration," in *Proc. IEEE Int. Conf. Computer Design (ICCD)*, 2024, tODO: authors.

[3] A. Bhattacharjee, A. Moitra, R. Yin, and P. Panda, "SITRA: Exploiting temporal silence in spiking transformers for fast & energy-efficient inference," in *Proc. IEEE/ACM Int. Symp. Low Power Electronics and Design (ISLPED)*, 2025, pp. 1–7.

[4] Z. Wang, F. Liu, X. Tang, and L. Jiang, "PS4: A low power SNN accelerator with spike speculative scheme," in *Proc. IEEE 42nd Int. Conf. Computer Design (ICCD)*, 2024, pp. 76–83.

[5] S. Matinizadeh, M. L. Varshika, and A. Das, "Optimizing memory latency and bandwidth of spiking neural network accelerators on FPGA via sparse hashing (SPARSH)," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2025, pp. 1–9.

[6] K. Lee and J. Park, "C-STEP: Compute-efficient spiking transformers with temporal exit and early-guided pruning," in *Proc. Design, Automation and Test in Europe (DATE)*, 2026.

[7] X. Cheng, S. Cao, S. Wang, M. Wang, W. Li, and X. Zeng, "An FPGA-based event-driven SNN accelerator for DVS applications with structured sparsity and early-stop," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 7, pp. 3298–3310, 2025.

[8] Z. Aizaz, J. C. Knight, and T. Nowotny, "FeNN-DMA: A RISC-V SoC for SNN acceleration," arXiv:2511.00732, 2025.

[9] I. Aliyev, J. Lopez, and T. Adegbija, "Surrogates, spikes, and sparsity: Performance analysis and characterization of SNN hyperparameters on hardware," arXiv:2603.24891, 2026.

[10] Z. Wu, M. Song, and H. K.-H. So, "Towards asynchronously triggered spiking neural network on FPGA for event-based vision," in *Proc. Int. Conf. Field-Programmable Technology (ICFPT)*, 2023, pp. 292–293.